

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – FUNCTION CALLING & STRUCTURED OUTPUT TASK

Student Name: chavali chandu
Reg. No.: 192472332
Course Title: Generative AI and Large Scale Models
Course Code: CSA6502

35. LLM Architecture and Function Calling A developer wants to understand how an LLM processes a request before generating a function call. Task: Explain the role of tokenization, embeddings, transformer layers, attention mechanisms, and output generation in the function-calling process. Represent the process using a suitable architecture diagram.

SOLUTION:

Introduction

Large Language Models (LLMs) such as **GPT-4**, **GPT-4o**, and **GPT-5** can understand user requests and decide whether to answer directly or call an external function (tool/API). Function calling allows the LLM to interact with databases, weather APIs, calculators, payment systems, or booking services while returning structured JSON outputs.

1. LLM Function Calling Process

Step 1: User Request

Example:

"Book a flight from Chennai to Delhi on 15 August."

The request is sent to the LLM.

↓

Step 2: Tokenization

The tokenizer breaks the sentence into small pieces called **tokens**.

Example:

["Book"]

["a"]

["flight"]

["from"]

["Chennai"]

["to"]

["Delhi"]

["on"]

["15"]

["August"]

Role:

- Converts text into machine-readable tokens.
- Reduces vocabulary complexity.

Step 3: Embeddings

Each token is converted into a high-dimensional numeric vector.

Example:

Book → [0.12, -0.53, ...]

Flight → [0.67, 0.91, ...]

Delhi → [0.31, 0.47, ...]

Role:

- Represents semantic meaning.
- Similar words have similar vector representations.

Step 4: Transformer Layers

The embeddings pass through multiple transformer layers.

Each layer performs:

- Multi-head attention
- Feed-forward neural network
- Layer normalization
- Residual connections

Role:

- Learns complex relationships.
- Understands sentence context.

Step 5: Attention Mechanism

Attention determines which words are most important.

Example:

Book

Flight

Chennai

Delhi

15 August

The model pays more attention to:

- flight
- Chennai
- Delhi
- date

instead of less important words like "a", "from", and "to".

Role:

- Captures contextual meaning.
- Identifies important entities for function calling.

Step 6: Function Selection

The model determines that a flight-booking function should be called.

Example function:

`book_flight()`

Parameters extracted:

Origin = Chennai

Destination = Delhi

Date = 15 August

Step 7: Structured Output (JSON)

Instead of generating free text, the LLM creates structured JSON.

Example:

```
{
  "function": "book_flight",
  "arguments": {
    "origin": "Chennai",
    "destination": "Delhi",
    "date": "2026-08-15"
  }
}
```

The application sends this JSON to the booking API.

Step 8: API Execution

The external API executes:

```
book_flight(
  origin="Chennai",
  destination="Delhi",
  date="2026-08-15"
)
```

↓

Booking confirmation returned.

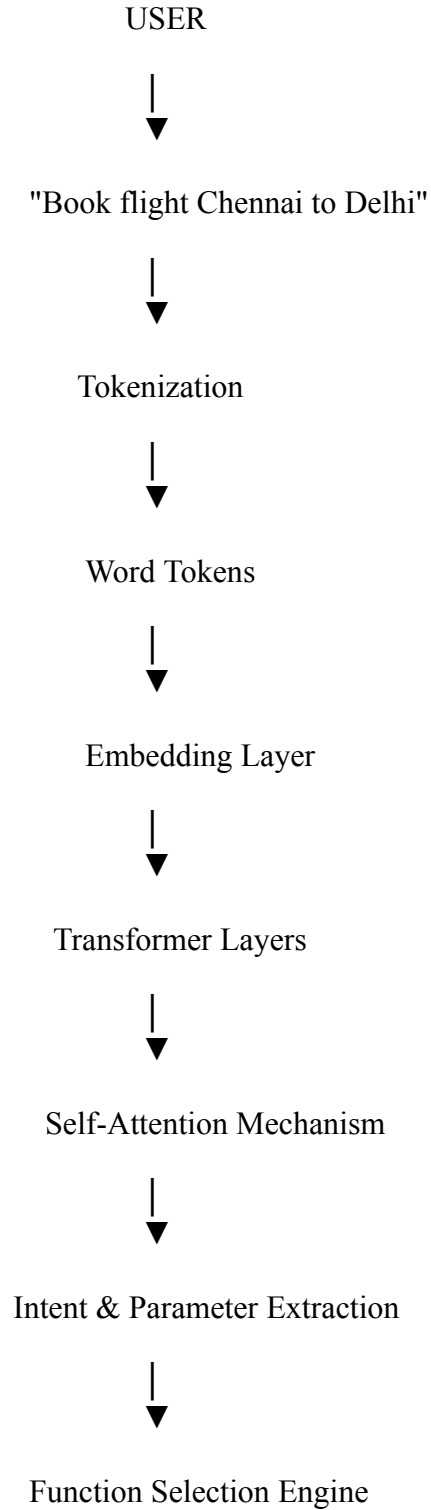
Step 9: Natural Language Response

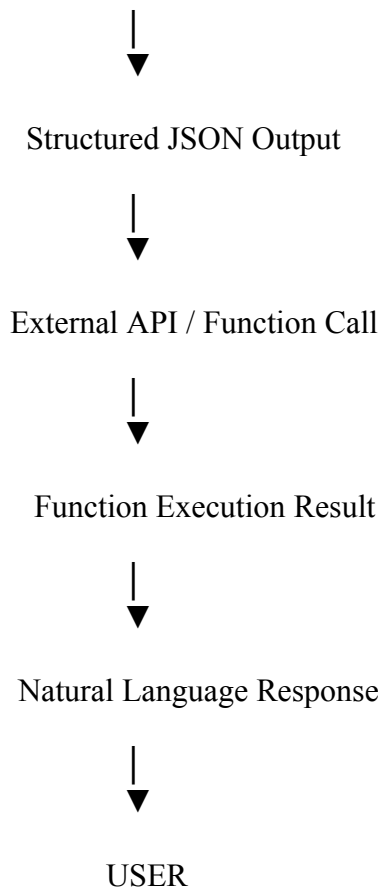
The LLM converts the API response into human language.

Example:

"Your flight from Chennai to Delhi on 15 August has been booked successfully."

2. Architecture Diagram





3. Function Calling

Function Calling enables an LLM to invoke external tools or APIs instead of generating answers solely from its internal knowledge.

Example

User:

"What's the weather in Chennai?"

LLM generates:

```
{
  "function": "get_weather",
  "arguments": {
    "city": "Chennai"
  }
}
```

The application calls:

`get_weather("Chennai")`

API returns:

Temperature = 33°C

Humidity = 72%

LLM replies:

"The current temperature in Chennai is 33°C with 72% humidity."

4. Structured Output

Structured Output ensures the LLM returns data in a predefined format (such as JSON or XML) that applications can process automatically.

Example 1: Weather

```
{  
  "city": "Chennai",  
  "temperature": 33,  
  "humidity": 72  
}
```

Example 2: Flight Booking

```
{  
  "origin": "Chennai",  
  "destination": "Delhi",  
  "date": "2026-08-15"  
}
```

Example 3: Hospital Appointment

```
{  
  "patient": "John",  
  "doctor": "Dr. Smith",  
  "date": "2026-08-20",  
  "time": "10:30 AM"  
}
```

Advantages of Function Calling

- Retrieves real-time information from external systems.
- Automates tasks such as booking, payments, and scheduling.
- Improves accuracy by using live data.
- Integrates seamlessly with APIs and databases.
- Reduces manual intervention.

Advantages of Structured Output

- Consistent and machine-readable format.
- Easy integration with software applications.
- Reduces parsing errors.
- Simplifies API communication.
- Supports automation and validation.

Conclusion

An LLM processes a user's request through **tokenization**, **embeddings**, **transformer layers**, and **attention mechanisms** to understand the intent and extract key information. When an external action is needed, it generates a **structured JSON output** that specifies the function name and arguments. The application executes the function or API call, and the LLM converts the result into a clear, natural-language response. This combination of **function calling** and **structured output** enables LLMs to interact reliably with real-world systems such as booking platforms, weather services, healthcare applications, and enterprise software.